Financial Fraud Detection using Python

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Step 2 : Importing the file

```
df = pd.read_csv(r"C:\Users\praty\cc_data.csv")
print("File imported successfully!")
```

```
File imported successfully!
```

Display the first 5 rows

```
df.head()
```

```
     index trans_date_trans_time              cc_num  \
0   151888      23-03-2019 03:06     3586008444788260
1  1185025      10-05-2020 11:28     4671727014157740
2    10818      07-01-2019 13:31      377993105397617
3   975275      30-01-2020 18:49  4710826438164840000
4   346814      12-06-2019 03:04      372382441451095

                          merchant       category     amt    first
last  \
0  fraud_Ferry, Lynch and Kautzer       misc_net    1.24  Crystal
Fuller
1              fraud_Bogisich Inc    grocery_pos  112.57  Kenneth
Edwards
2                fraud_Mohr-Bayer   shopping_net    6.69   Nathan
Martinez
3             fraud_Langworth LLC  personal_care  100.69     Juan
Henry
4             fraud_Hills-Witting   shopping_net    7.80    Brent
Terrell

  gender                       street  ...      lat      long  city_pop
\
0      F          000 Jennifer Mills  ...  47.4974 -122.0107     50489

1      M         3653 Ryan Crossroad  ...  40.8618  -85.6067      2304
```

```
2      M           586 Thomas Cliffs  ...  44.8755  -88.1555      5548

3      M  9795 Lori Island Suite 346  ...  48.8328 -108.3961       192

4      M     502 Rice Plaza Apt. 979  ...  41.8948  -73.9767      2471


                            job         dob
trans_num  \
0                     Osteopath  29-02-1984
ff66e2310bf7a2a05e65d6b0a363aa5b
1                Retail banker  25-07-1955
f1edcfa2102065f5f76375c2d20b9081
2              Mining engineer  11-09-1975
0caa0ca338d1d2903b92bb42d8905768
3  Further education lecturer  04-01-1964
c7d25a5388311ebd7f2eef77459376ac
4             Heritage manager  21-01-1966
8e1306b47052ac10fe48512de79edf71

    unix_time  merch_lat  merch_long  is_fraud
0  1332471978  47.352172 -121.644080         0
1  1368185292  40.265726  -85.914050         0
2  1325943086  44.512954  -88.693464         0
3  1359571786  47.909634 -109.008657         0
4  1339470288  41.534838  -74.449927         0

[5 rows x 23 columns]
```

Step 3: Initial Data Inspection

```
df.shape
```

```
(389002, 23)
```

```
df.columns
```

```
Index(['index', 'trans_date_trans_time', 'cc_num', 'merchant',
'category',
       'amt', 'first', 'last', 'gender', 'street', 'city', 'state',
'zip',
       'lat', 'long', 'city_pop', 'job', 'dob', 'trans_num',
'unix_time',
       'merch_lat', 'merch_long', 'is_fraud'],
      dtype='str')
```

Dataset Information

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 389002 entries, 0 to 389001
Data columns (total 23 columns):
 #   Column                  Non-Null Count   Dtype  
---  ------                  --------------   -----  
 0   index                   389002 non-null  int64  
 1   trans_date_trans_time   389002 non-null  str    
 2   cc_num                  389002 non-null  int64  
 3   merchant                389002 non-null  str    
 4   category                389002 non-null  str    
 5   amt                     389002 non-null  float64
 6   first                   389002 non-null  str    
 7   last                    389002 non-null  str    
 8   gender                  389002 non-null  str    
 9   street                  389002 non-null  str    
 10  city                    389002 non-null  str    
 11  state                   389002 non-null  str    
 12  zip                     389002 non-null  int64  
 13  lat                     389002 non-null  float64
 14  long                    389002 non-null  float64
 15  city_pop                389002 non-null  int64  
 16  job                     389002 non-null  str    
 17  dob                     389002 non-null  str    
 18  trans_num               389002 non-null  str    
 19  unix_time               389002 non-null  int64  
 20  merch_lat               389002 non-null  float64
 21  merch_long              389002 non-null  float64
 22  is_fraud                389002 non-null  int64  
dtypes: float64(5), int64(6), str(12)
memory usage: 68.3 MB
```

```
df.isnull().sum()
```

```
index                     0
trans_date_trans_time     0
cc_num                    0
merchant                  0
category                  0
amt                       0
first                     0
last                      0
gender                    0
street                    0
city                      0
state                     0
zip                       0
lat                       0
long                      0
city_pop                  0
job                       0
```

```
dob                          0
trans_num                    0
unix_time                    0
merch_lat                    0
merch_long                   0
is_fraud                     0
dtype: int64
```

```
categorical_columns = df.select_dtypes(include='object').columns

for column in categorical_columns:
    print(f"{column}: {df[column].nunique()}")
```

```
C:\Users\praty\AppData\Local\Temp\ipykernel_36664\2379879581.py:1:
Pandas4Warning: For backward compatibility, 'str' dtypes are included
by select_dtypes when 'object' dtype is specified. This behavior is
deprecated and will be removed in a future version. Explicitly pass
'str' to `include` to select them, or to `exclude` to remove them and
silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-
strings.html#string-migration-select-dtypes for details on how to
write code that works with pandas 2 and 3.
  categorical_columns = df.select_dtypes(include='object').columns

trans_date_trans_time: 293627
merchant: 693
category: 14
first: 352
last: 481
gender: 2
street: 979
city: 890
state: 51
job: 492
dob: 964
trans_num: 389002
```

What is the distribution of numerical variables in the dataset?

```
# Select all numerical columns
numerical_columns = df.select_dtypes(include=['int64',
'float64']).columns

print(numerical_columns)
```

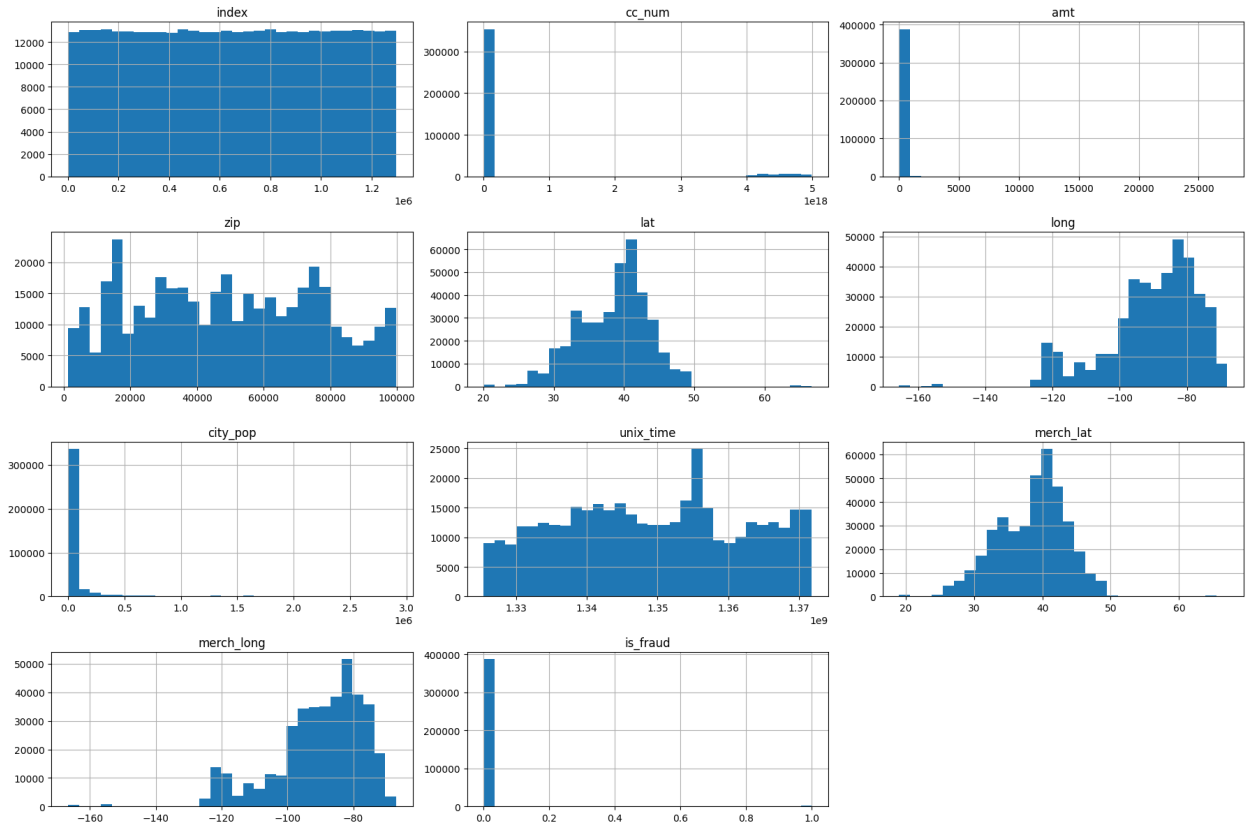
```
Index(['index', 'cc_num', 'amt', 'zip', 'lat', 'long', 'city_pop',
'unix_time',
```

```
       'merch_lat', 'merch_long', 'is_fraud'],
      dtype='str')

# Plot histograms for all numerical variables

df[numerical_columns].hist(figsize=(18,12), bins=30)

plt.tight_layout()
plt.show()
```



Summary Statistics

```
df.describe().T
```

```
                count          mean           std           min
25%  \
index        389002.0  6.485205e+05  3.745744e+05  1.100000e+01
3.241842e+05
cc_num       389002.0  4.191512e+17  1.311579e+18  6.041621e+10
1.800429e+14
amt          389002.0  7.044215e+01  1.622039e+02  1.000000e+00
9.660000e+00
zip          389002.0  4.881806e+04  2.687938e+04  1.257000e+03
2.623700e+04
lat          389002.0  3.853312e+01  5.074596e+00  2.002710e+01
```

```
3.462050e+01
long        389002.0 -9.023766e+01  1.374586e+01 -1.656723e+02 -
9.679800e+01
city_pop    389002.0  8.868084e+04  3.012101e+05  2.300000e+01
7.430000e+02
unix_time   389002.0  1.349251e+09  1.285085e+07  1.325376e+09
1.338751e+09
merch_lat   389002.0  3.853168e+01  5.109400e+00  1.902980e+01
3.471939e+01
merch_long  389002.0 -9.023667e+01  1.375731e+01 -1.666696e+02 -
9.690544e+01
is_fraud    389002.0  5.789173e-03  7.586616e-02  0.000000e+00
0.000000e+00

                      50%           75%           max
index       6.486485e+05  9.735032e+05  1.296674e+06
cc_num      3.521417e+15  4.642255e+15  4.992346e+18
amt         4.757000e+01  8.307750e+01  2.739012e+04
zip         4.817400e+04  7.201100e+04  9.978300e+04
lat         3.935430e+01  4.194040e+01  6.669330e+01
long       -8.747690e+01 -8.015800e+01 -6.795030e+01
city_pop    2.456000e+03  2.032800e+04  2.906700e+06
unix_time   1.349267e+09  1.359460e+09  1.371817e+09
merch_lat   3.936107e+01  4.195601e+01  6.706428e+01
merch_long -8.744684e+01 -8.025383e+01 -6.695654e+01
is_fraud    0.000000e+00  0.000000e+00  1.000000e+00
```

Correlation Analysis

```
# Select only numerical columns
numerical_columns = df.select_dtypes(include=['int64', 'float64'])

# Compute the correlation matrix
correlation_matrix = numerical_columns.corr()

correlation_matrix
```

```
               index    cc_num       amt       zip       lat      long
\
index       1.000000  0.001289  0.000779  0.000874 -0.000193 -0.001494

cc_num      0.001289  1.000000  0.002975  0.040486 -0.059182 -0.047587

amt         0.000779  0.002975  1.000000  0.001756 -0.000445 -0.000517

zip         0.000874  0.040486  0.001756  1.000000 -0.112567 -0.910295

lat        -0.000193 -0.059182 -0.000445 -0.112567  1.000000 -0.016080

long       -0.001494 -0.047587 -0.000517 -0.910295 -0.016080  1.000000
```

```
city_pop   -0.001800 -0.009125  0.007271  0.078373 -0.155972 -0.053461

unix_time   0.998976  0.001333  0.000780  0.000905 -0.000150 -0.001608

merch_lat  -0.000383 -0.058560 -0.000314 -0.111689  0.993599 -0.016010

merch_long -0.001475 -0.047549 -0.000578 -0.909470 -0.015994  0.999119

is_fraud   -0.006121 -0.001280  0.210706 -0.001220  0.002643  0.001376
```

	city_pop	unix_time	merch_lat	merch_long	is_fraud
index	-0.001800	0.998976	-0.000383	-0.001475	-0.006121
cc_num	-0.009125	0.001333	-0.058560	-0.047549	-0.001280
amt	0.007271	0.000780	-0.000314	-0.000578	0.210706
zip	0.078373	0.000905	-0.111689	-0.909470	-0.001220
lat	-0.155972	-0.000150	0.993599	-0.015994	0.002643
long	-0.053461	-0.001608	-0.016010	0.999119	0.001376
city_pop	1.000000	-0.001766	-0.154767	-0.053468	0.001176
unix_time	-0.001766	1.000000	-0.000354	-0.001582	-0.006562
merch_lat	-0.154767	-0.000354	1.000000	-0.015932	0.002406
merch_long	-0.053468	-0.001582	-0.015932	1.000000	0.001351
is_fraud	0.001176	-0.006562	0.002406	0.001351	1.000000

```
plt.figure(figsize=(12,8))

plt.imshow(correlation_matrix, cmap='coolwarm', interpolation='none')

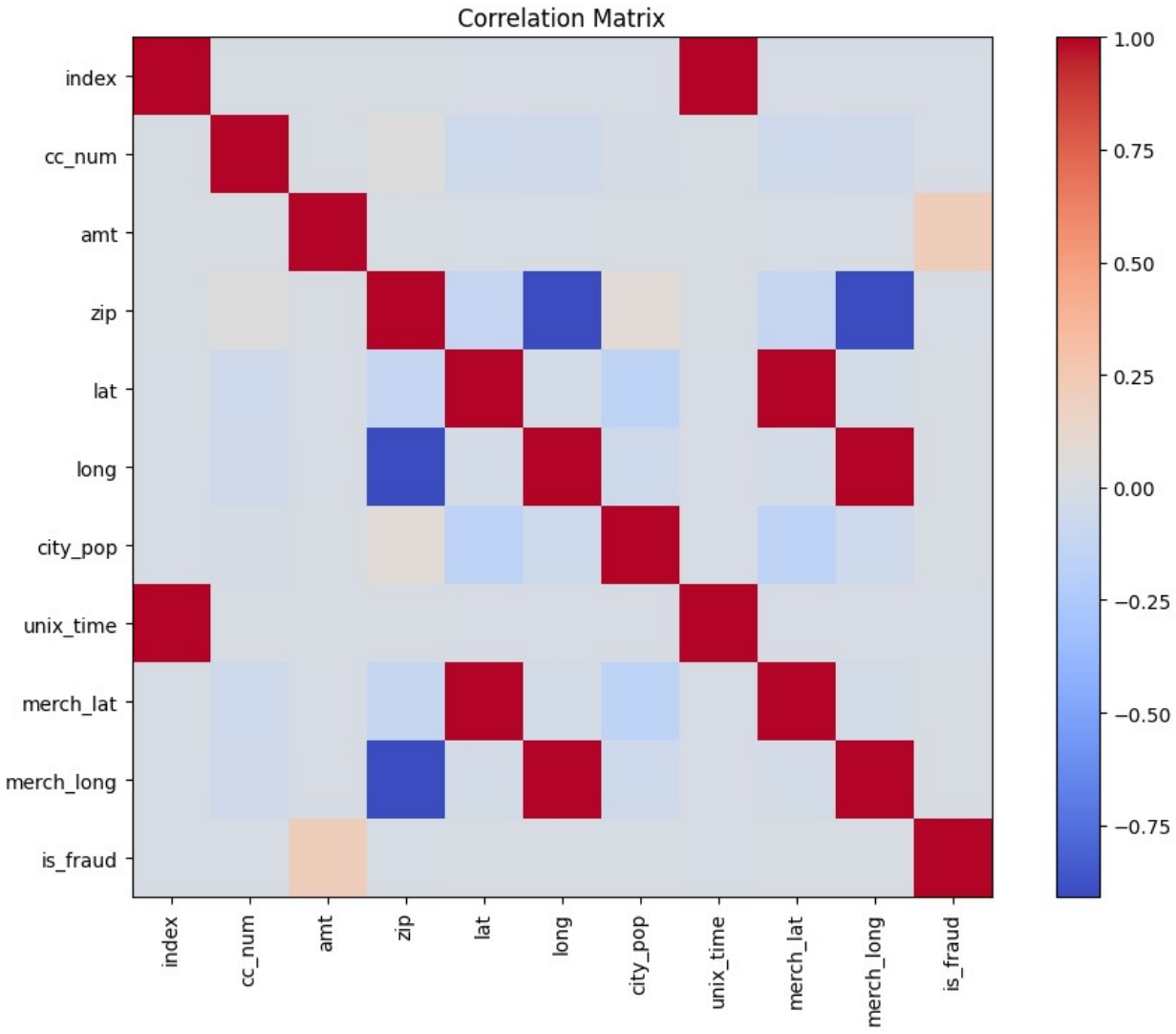
plt.colorbar()

plt.xticks(range(len(correlation_matrix.columns)),
           correlation_matrix.columns,
           rotation=90)

plt.yticks(range(len(correlation_matrix.columns)),
           correlation_matrix.columns)

plt.title("Correlation Matrix")

plt.show()
```



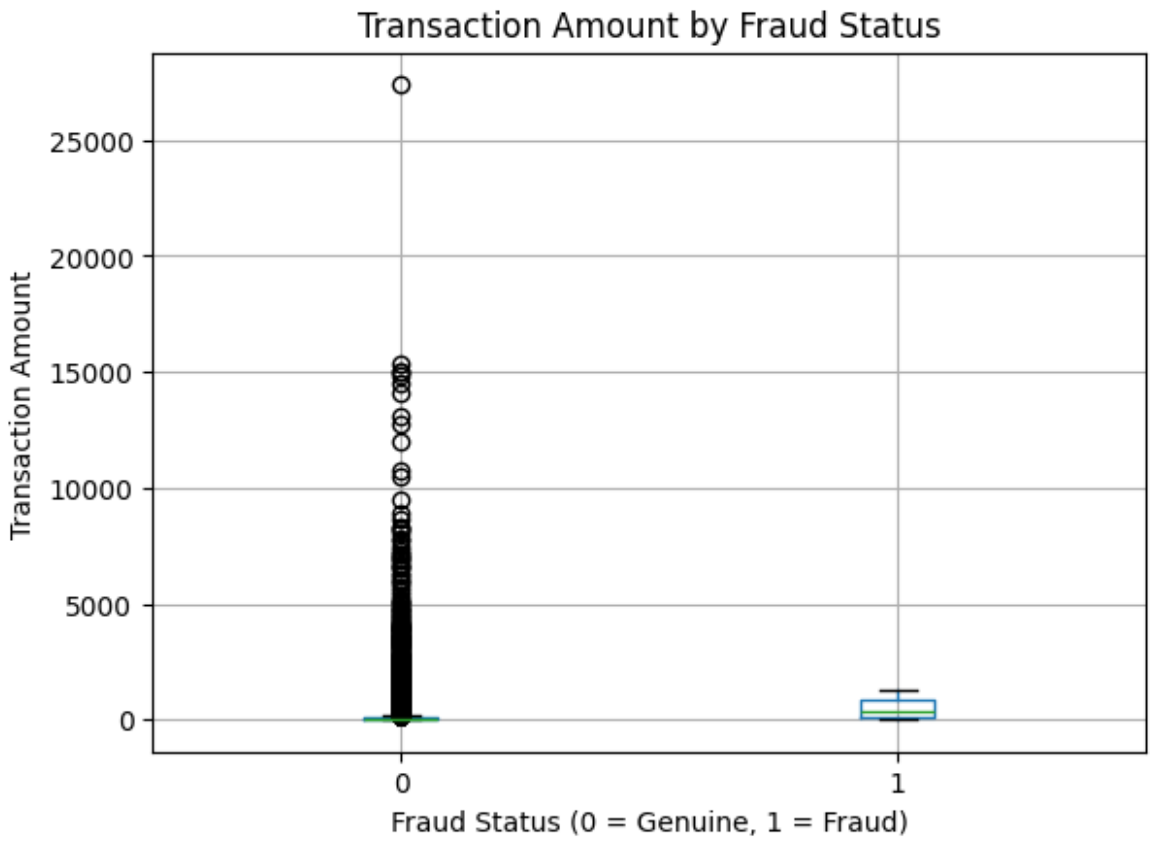
Distribution of Transaction Amount by Fraud Status

```
plt.figure(figsize=(10,6))

df.boxplot(column='amt', by='is_fraud')

plt.title("Transaction Amount by Fraud Status")
plt.suptitle("")
plt.xlabel("Fraud Status (0 = Genuine, 1 = Fraud)")
plt.ylabel("Transaction Amount")
plt.show()
df.groupby("is_fraud")["amt"].describe()
```

```
<Figure size 1000x600 with 0 Axes>
```




```
              count         mean          std    min        25%       50%
75%  \
is_fraud

0          386750.0    67.834157   156.227752   1.00     9.6200    47.330
82.5000
1            2252.0   518.328677   389.127281   1.18   147.8825   365.285
888.3425

                max
is_fraud
0          27390.12
1           1294.83
```

```
plt.figure(figsize=(10,6))

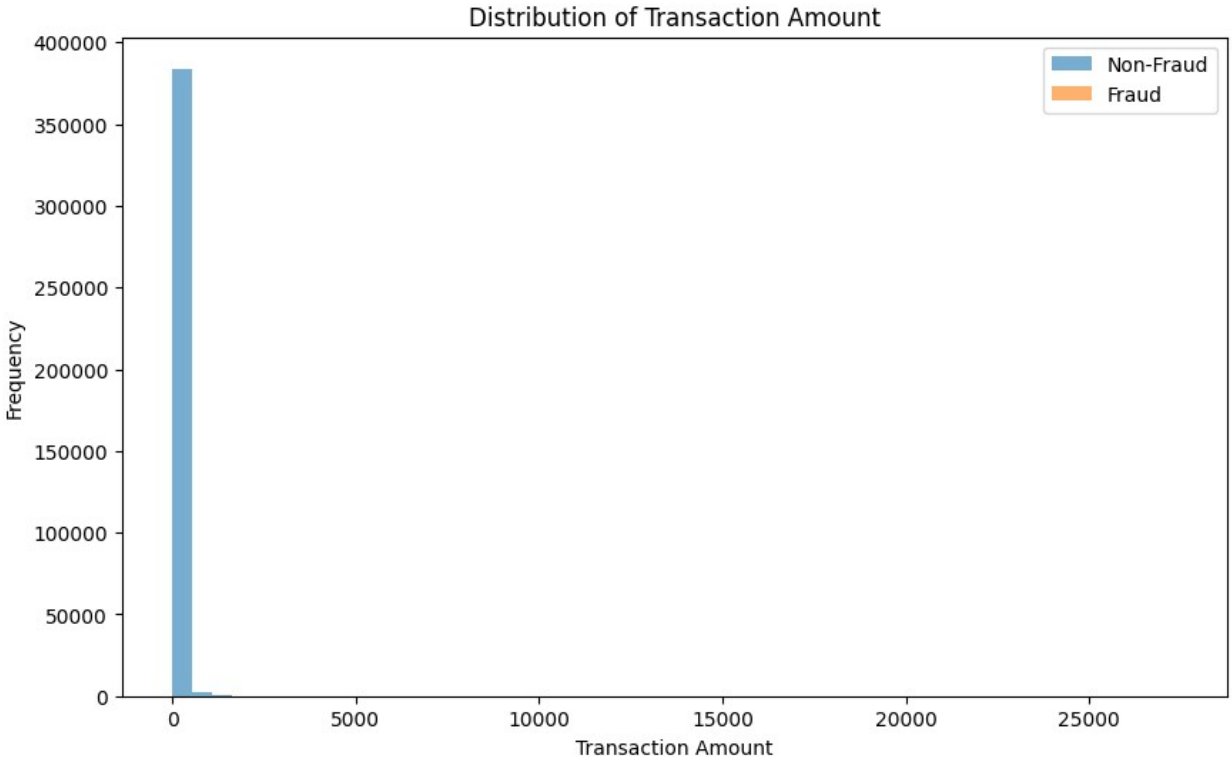
plt.hist(df[df['is_fraud']==0]['amt'],
         bins=50,
         alpha=0.6,
         label='Non-Fraud')

plt.hist(df[df['is_fraud']==1]['amt'],
         bins=50,
         alpha=0.6,
```

```
         label='Fraud')

plt.xlabel("Transaction Amount")
plt.ylabel("Frequency")
plt.title("Distribution of Transaction Amount")
plt.legend()

plt.show()
```



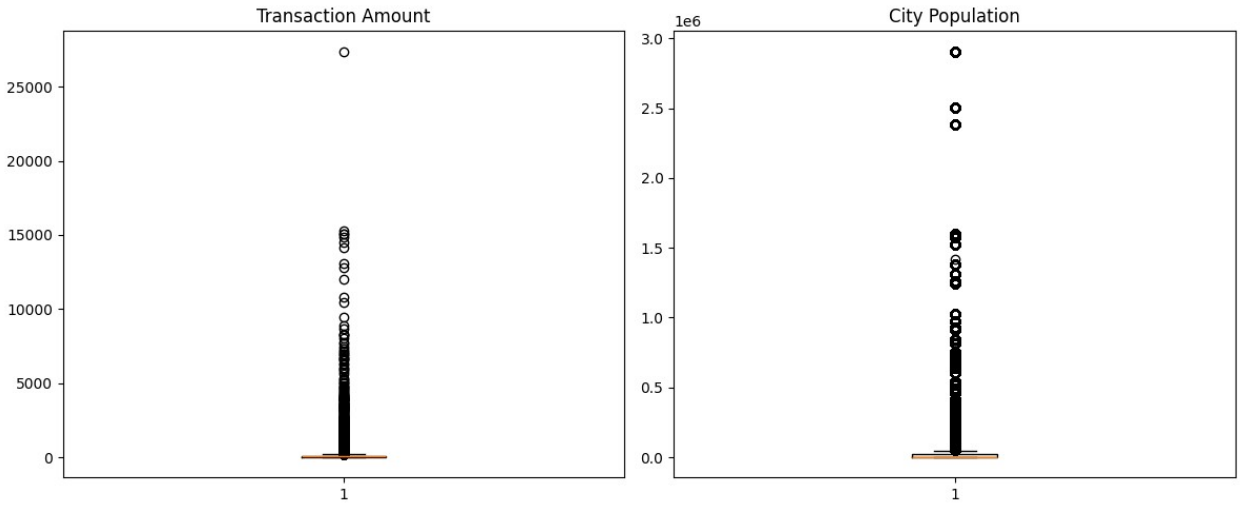
Outlier Detection

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.boxplot(df['amt'])
plt.title("Transaction Amount")

plt.subplot(1,2,2)
plt.boxplot(df['city_pop'])
plt.title("City Population")

plt.tight_layout()
plt.show()
```

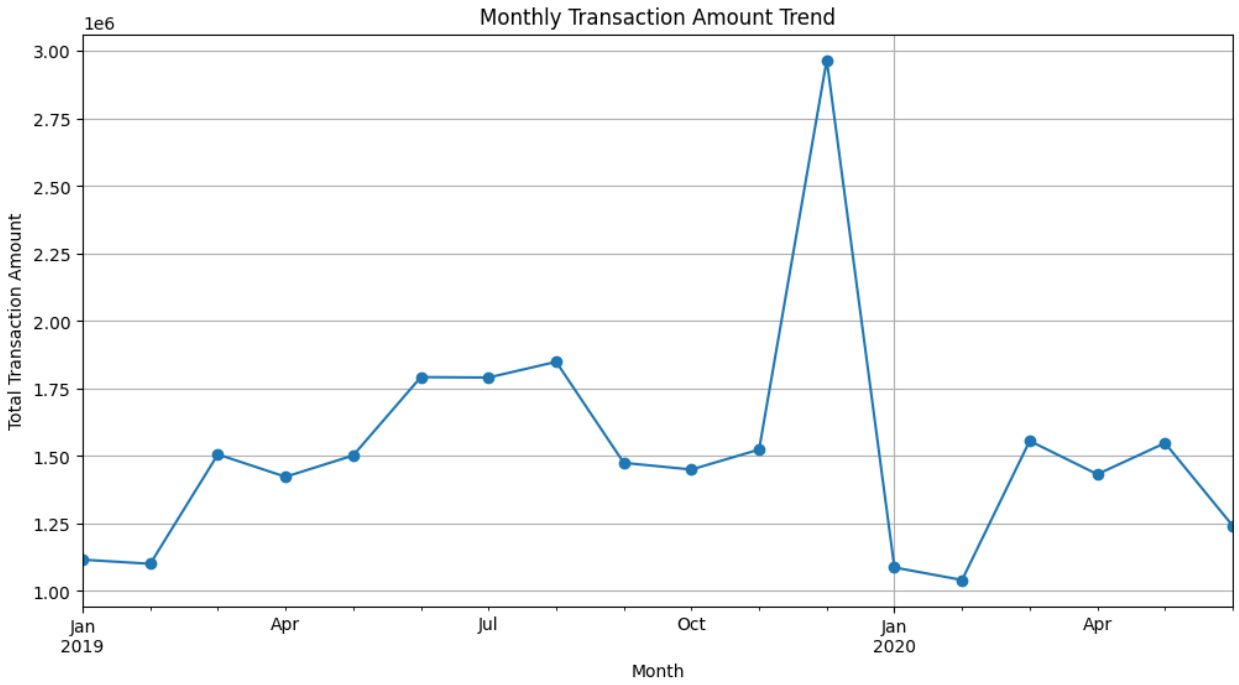


Trends over time

```
monthly_amount.plot(
    kind='line',
    figsize=(12,6),
    marker='o'
)

plt.title("Monthly Transaction Amount Trend")
plt.xlabel("Month")
plt.ylabel("Total Transaction Amount")
plt.grid(True)

plt.show()
```

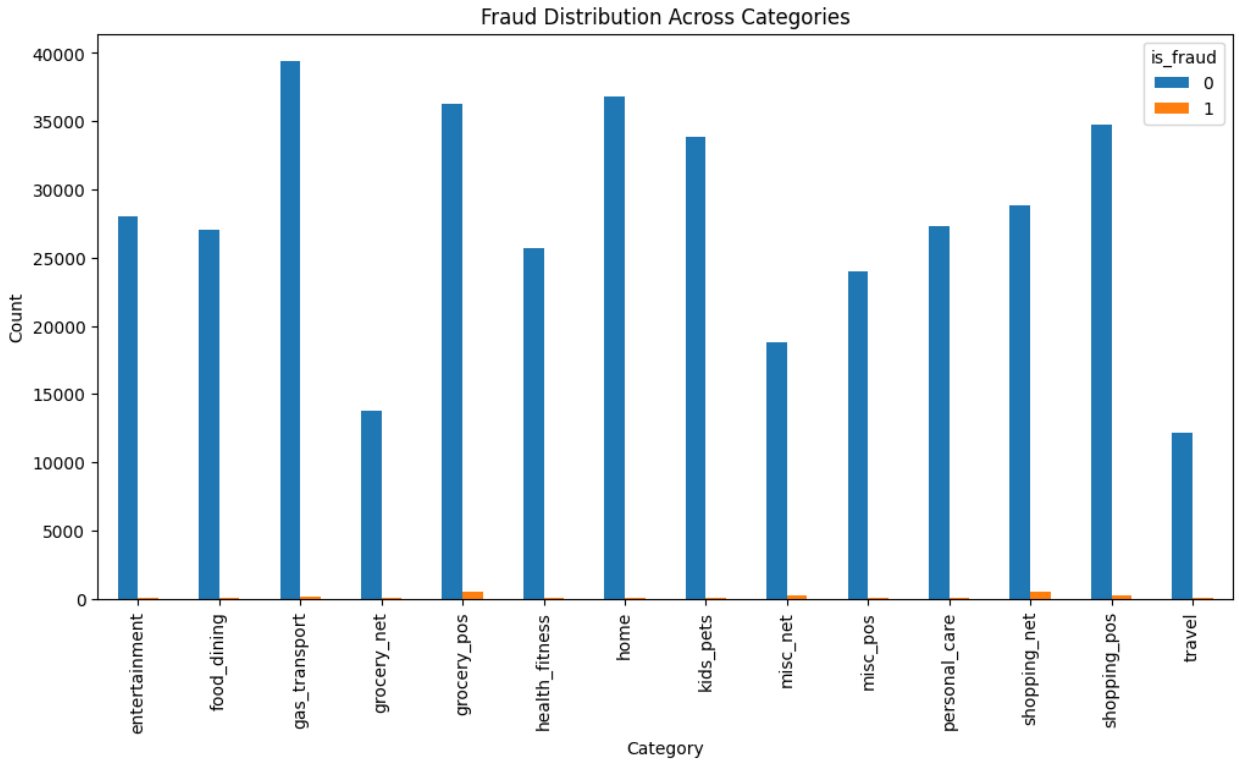


Distribution of target variable across categories

```
fraud_category = pd.crosstab(df['category'], df['is_fraud'])

fraud_category.plot(kind='bar',
                    figsize=(12,6))

plt.title("Fraud Distribution Across Categories")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```



Distribution of Fraud Across Categories

```
fraud_category = pd.crosstab(df['category'], df['is_fraud'])

fraud_category
```

is_fraud	0	1
category		
entertainment	28054	68
food_dining	27028	42
gas_transport	39440	193
grocery_net	13806	40
grocery_pos	36245	518
health_fitness	25693	39
home	36832	65
kids_pets	33833	74
misc_net	18825	287
misc_pos	23971	77
personal_care	27297	76
shopping_net	28816	478
shopping_pos	34759	253
travel	12151	42

Unusal Values

```
df.describe(include='all').T
```

```
                           count  unique                                 top  \
index                   389002.0     NaN                                 NaN
trans_date_trans_time    389002     NaN                                 NaN
cc_num                  389002.0     NaN                                 NaN
merchant                 389002     693                  fraud_Kilback LLC
category                 389002      14                     gas_transport
amt                     389002.0     NaN                                 NaN
first                    389002     352                       Christopher
last                     389002     481                             Smith
gender                   389002       2                                 F
street                   389002     979                  1652 James Mews
city                     389002     890                        Birmingham
state                    389002      51                                TX
zip                     389002.0     NaN                                 NaN
lat                     389002.0     NaN                                 NaN
long                    389002.0     NaN                                 NaN
city_pop                389002.0     NaN                                 NaN
job                      389002     492                  Film/video editor
dob                      389002     964                         23-03-1977
trans_num                389002  389002  ff66e2310bf7a2a05e65d6b0a363aa5b
unix_time               389002.0     NaN                                 NaN
merch_lat               389002.0     NaN                                 NaN
merch_long              389002.0     NaN                                 NaN
is_fraud                389002.0     NaN                                 NaN

                            freq                          mean  \
```

index	NaN	648520.514213
trans_date_trans_time	NaN	2019-10-03 14:43:26.992457
cc_num	NaN	419151203195879616.0
merchant	1305	NaN
category	39633	NaN
amt	NaN	70.442148
first	7982	NaN
last	8511	NaN
gender	213231	NaN
street	980	NaN
city	1740	NaN
state	28482	NaN
zip	NaN	48818.064295
lat	NaN	38.533121
long	NaN	-90.237664
city_pop	NaN	88680.842857
job	2945	NaN
dob	1694	NaN
trans_num	1	NaN
unix_time	NaN	1349250578.548671
merch_lat	NaN	38.531683
merch_long	NaN	-90.236674
is_fraud	NaN	0.005789

	min	25%	\
index	11.0	324184.25	
trans_date_trans_time	2019-01-01 00:06:00	2019-06-03 19:15:15	
cc_num	60416207185.0	180042946491150.0	
merchant	NaN	NaN	
category	NaN	NaN	
amt	1.0	9.66	
first	NaN	NaN	
last	NaN	NaN	
gender	NaN	NaN	
street	NaN	NaN	
city	NaN	NaN	
state	NaN	NaN	
zip	1257.0	26237.0	
lat	20.0271	34.6205	
long	-165.6723	-96.798	
city_pop	23.0	743.0	
job	NaN	NaN	
dob	NaN	NaN	
trans_num	NaN	NaN	
unix_time	1325376413.0	1338750920.0	
merch_lat	19.029798	34.719394	
merch_long	-166.669638	-96.905445	
is_fraud	0.0	0.0	

	50%	75%	\

index	648648.5	973503.25
trans_date_trans_time	2019-10-03 12:22:00	2020-01-29 11:37:45
cc_num	3521417320836160.0	4642255475285940.0
merchant	NaN	NaN
category	NaN	NaN
amt	47.57	83.0775
first	NaN	NaN
last	NaN	NaN
gender	NaN	NaN
street	NaN	NaN
city	NaN	NaN
state	NaN	NaN
zip	48174.0	72011.0
lat	39.3543	41.9404
long	-87.4769	-80.158
city_pop	2456.0	20328.0
job	NaN	NaN
dob	NaN	NaN
trans_num	NaN	NaN
unix_time	1349266968.0	1359459514.25
merch_lat	39.361065	41.956012
merch_long	-87.446843	-80.253831
is_fraud	0.0	0.0

	max	std
index	1296674.0	374574.390202
trans_date_trans_time	2020-06-21 12:13:00	NaN
cc_num	4992346398065149952.0	1311578663900893440.0
merchant	NaN	NaN
category	NaN	NaN
amt	27390.12	162.203915
first	NaN	NaN
last	NaN	NaN
gender	NaN	NaN
street	NaN	NaN
city	NaN	NaN
state	NaN	NaN
zip	99783.0	26879.383224
lat	66.6933	5.074596
long	-67.9503	13.745855
city_pop	2906700.0	301210.101718
job	NaN	NaN
dob	NaN	NaN
trans_num	NaN	NaN
unix_time	1371816817.0	12850848.628278
merch_lat	67.064277	5.1094
merch_long	-66.95654	13.757311
is_fraud	1.0	0.075866

Are there any potential data entry errors or inconsistencies?

```
df.duplicated().sum()
```

```
np.int64(0)
```

How does the distribution of numerical variables vary between different groups or segments of the dataset?

```
df.groupby('gender')['amt'].describe()

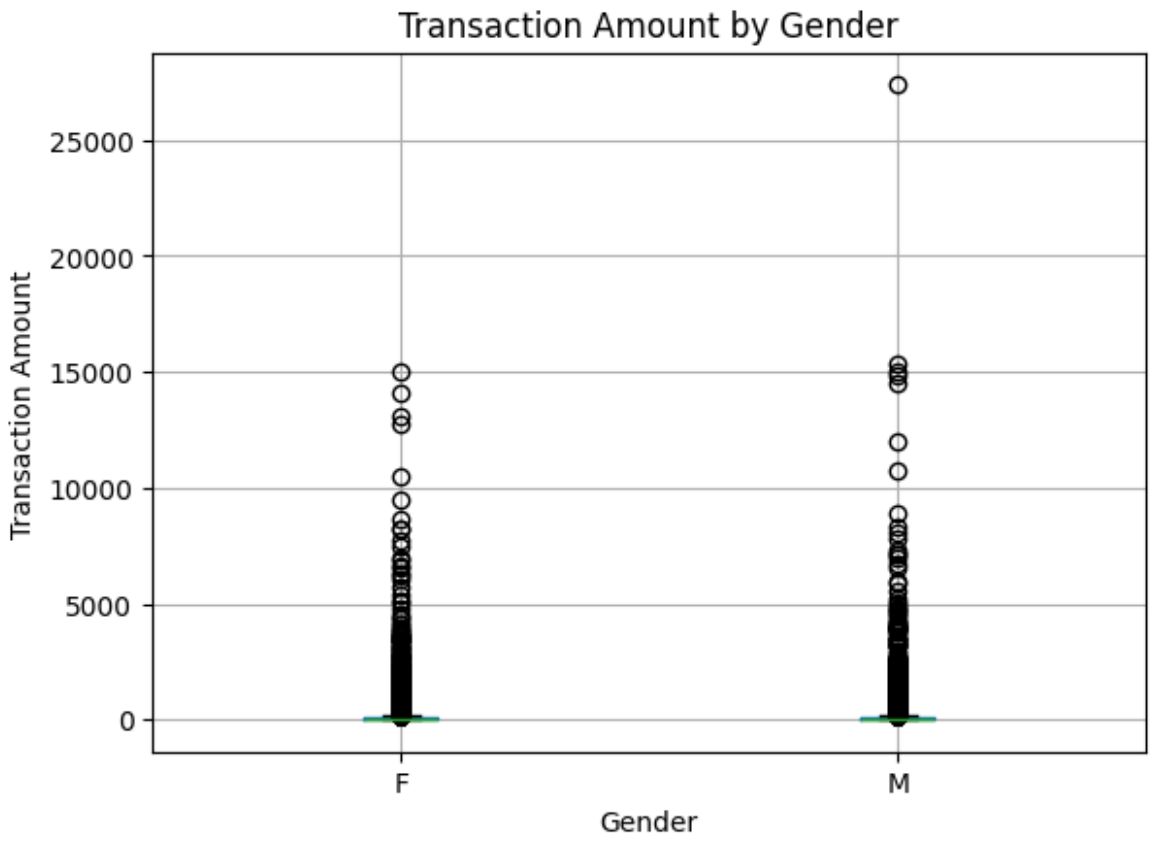
plt.figure(figsize=(8,6))

df.boxplot(column='amt', by='gender')

plt.title("Transaction Amount by Gender")
plt.suptitle("")
plt.xlabel("Gender")
plt.ylabel("Transaction Amount")

plt.show()
```

```
<Figure size 800x600 with 0 Axes>
```



Top factors influencing fraud

```
fraud_analysis = (
    df.drop(columns=['index', 'cc_num'])
      .groupby('is_fraud')
      .mean(numeric_only=True)
      .T
)

fraud_analysis
```

is_fraud	0	1
amt	6.783416e+01	5.183287e+02
zip	4.882057e+04	4.838817e+04
lat	3.853210e+01	3.870890e+01
long	-9.023911e+01	-8.998971e+01
city_pop	8.865381e+04	9.332321e+04
unix_time	1.349257e+09	1.348145e+09
merch_lat	3.853074e+01	3.869280e+01
merch_long	-9.023809e+01	-8.999310e+01

Exploratory Data Analysis (EDA) Report

Financial Fraud Detection using Python

Introduction

The objective of this Exploratory Data Analysis (EDA) is to analyze credit card transaction data and identify patterns, trends, and anomalies that may help detect fraudulent transactions. The analysis provides insights into the structure, quality, and characteristics of the dataset before developing predictive models for fraud detection.

Dataset Overview

The dataset contains credit card transaction records with customer information, merchant details, transaction amount, geographical information, and fraud status (`is_fraud`). Python libraries including **Pandas**, **NumPy**, and **Matplotlib** were used to perform the analysis.

Data Exploration

Dataset Dimensions

The dataset was successfully loaded and explored using the following functions:

- `df.shape`
- `df.head()`
- `df.tail()`
- `df.info()`

These functions provided information about the number of rows, columns, data types, and overall dataset structure.

Summary Statistics

Summary statistics were generated using:

```
df.describe().T
```

The analysis provided descriptive measures including:

- Count
- Mean
- Standard Deviation
- Minimum Value
- Maximum Value
- Quartiles (25%, 50%, and 75%)

These statistics helped understand the distribution and spread of numerical variables such as transaction amount and city population.

Categorical Variables

The number of unique values for every categorical variable was calculated using:

```
df[column].nunique()
```

This helped identify the diversity of:

- Merchants
- Transaction Categories
- Cities
- States
- Occupations
- Customer Names

Data Quality Assessment

Missing Values

Missing values were checked using:

```
df.isnull().sum()
```

Observation

No missing values were found in the dataset. Therefore, no missing value treatment was required.

Duplicate Records

Duplicate records were identified using:

```
df.duplicated().sum()
```

Observation

No duplicate transactions were found, indicating good data quality.

Data Types

The data types of all variables were verified using:

```
df.info()
```

The transaction date column was converted into datetime format to enable time-based analysis.

Distribution of Numerical Variables

Histograms were created for all numerical variables to understand their distributions.

Observation

- Most numerical variables are positively skewed.
- Transaction amounts are concentrated at lower values with relatively few high-value transactions.
- City population also contains large variations across different cities.

Correlation Analysis

Correlation analysis was performed using:

```
correlation_matrix = numerical_columns.corr()
```

The results were visualized using a correlation heatmap.

Observation

Most numerical variables showed weak linear relationships with one another, indicating that multiple independent variables contribute to the dataset.

Fraud Analysis

Transaction Amount vs Fraud Status

Boxplots and histograms were used to compare transaction amounts for genuine and fraudulent transactions.

Observation

- Fraudulent transactions generally have higher transaction amounts than genuine transactions.
- Genuine transactions contain several high-value outliers, indicating that large transactions are not always fraudulent.
- Transaction amount appears to be an important feature for fraud detection.

Outlier Detection

Outliers were detected using:

- Boxplots
- Interquartile Range (IQR) Method

The analysis was performed on:

- Transaction Amount (`amt`)
- City Population (`city_pop`)

Observation

Several outliers were identified, particularly in transaction amounts. These are expected in financial datasets because of occasional high-value transactions.

Fraud Distribution Across Categories

The distribution of fraudulent transactions across different transaction categories was analyzed.

Observation

Certain transaction categories experienced relatively higher fraud occurrences, suggesting that transaction type may contribute to identifying fraudulent behavior.

Time Series Analysis

Transaction dates were converted into datetime format and monthly transaction trends were analyzed.

Observation

The analysis provided insights into transaction behavior over time and highlighted overall spending trends.

Feature Comparison

The average values of numerical variables were compared between fraudulent and genuine transactions.

Observation

Among all numerical variables:

- **Transaction Amount (`amt`)** showed the largest difference between fraudulent and genuine transactions.
- Fraudulent transactions had a significantly higher average transaction amount.
- Other variables such as city population, latitude, longitude, and Unix timestamp exhibited relatively smaller differences.

Key Findings

- The dataset contains no missing values.
- No duplicate records were identified.
- Transaction amounts are highly right-skewed.
- Fraudulent transactions generally involve higher transaction amounts.
- Transaction amount appears to be one of the strongest numerical indicators of fraud.
- Most numerical variables exhibit weak correlations.
- Fraud frequency varies across different transaction categories.

Conclusion

The exploratory data analysis provided valuable insights into the financial transaction dataset. The data was found to be clean, consistent, and suitable for further analysis. Several important

patterns were identified, particularly the relationship between transaction amount and fraudulent activity.

The insights obtained through EDA provide a strong foundation for building fraud detection models and support data-driven decision-making in financial risk management.

Tools Used

- Python
- Pandas
- NumPy
- Matplotlib
- Jupyter Notebook

Outcome

The dataset has been successfully explored and validated. The analysis identified important trends, outliers, fraud patterns, and feature relationships that can be used in future machine learning models for financial fraud detection.